

FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Department of Artificial Intelligence & Data Science · Karachi Campus

PROGRAMMING FOR ARTIFICIAL INTELLIGENCE

Laboratory Manual

LAB 06

File Handling, Exceptions and Web APIs

Text Files · CSV · JSON · Paths

try / except / else / finally · requests · Status Codes · Parsing Live Data

PREPARED BY

Khadeejah Ashraf

Lab Instructor | Department of Artificial Intelligence & Data Science

Contents

1. Lab Objectives
2. Lab Rules and Conduct
3. Why Programs Need Files
4. Opening a File: `open()`, `with`, and the Modes
5. Reading Text Files
6. Writing Text Files
7. Exception Handling
8. Where Your Files Live: Paths
9. CSV Files
10. JSON Files
11. Web APIs and the `requests` Library
12. A Complete Pipeline: Fetch, Parse, Store
13. Common Errors
14. Quick Reference
15. Class Tasks (to be completed during the session)
16. Home Tasks
17. Submission and Assessment

1. Lab Objectives

By the end of this session, you should be able to:

- Explain why data held only in variables is lost when the program ends, and what a file gives you instead.
- Open a file with `with open(...)`, choose the correct mode, and say why the `with` form is preferred over a bare `open()`.
- Read a text file whole, as a list of lines, and one line at a time, and explain when each is appropriate.
- Write and append text, and control line endings correctly.
- Handle errors with `try / except / else / finally`, catch specific exception classes, and read the error object with `as err`.
- Raise an exception deliberately, and define a simple exception class of your own.
- Build file paths that work on any machine, and test for a file's existence.
- Read and write CSV files with `csv.reader`, `csv.writer`, `DictReader` and `DictWriter`.
- Convert between Python objects and JSON text with `json.dump`, `json.load`, `json.dumps` and `json.loads`, and state which Python types survive the trip.
- Explain what a web API is, send a GET request with `requests`, check the status code, and parse the JSON body into Python objects.
- Combine all of the above into a small pipeline that fetches live data, handles failure, and stores the result.

2. Lab Rules and Conduct

- Maintain discipline throughout the lab session.
- Listen carefully and follow the instructions as they are given.
- Raise your hand if you face any problem; do not leave your seat.
- **Class tasks** must be completed and demonstrated during the session itself.
- **Home tasks** must be submitted before the deadline announced on the course portal.
- Get your work checked by the instructor before the end of the session.

Reminder: work that is not checked during the session carries no marks, even if the code runs correctly.

3. Why Programs Need Files

Everything your programs have produced so far has lived in memory: a list of marks, a dictionary of word counts, a `Student` object. When the program ends, all of it disappears. A **file** is storage that outlives the program, and it is also the way two programs exchange data — the CSV your model reads next month was written by a program that finished running long ago.

Three formats cover almost everything you will meet in this course. They hold the same information; they differ in how much structure the file itself records.

marks.txt	marks.csv	marks.json
<pre>Ali 78 Sara 92 Bilal 65</pre>	<pre>name,mark Ali,78 Sara,92 Bilal,65</pre>	<pre>[{"name": "Ali", "mark": 78}, {"name": "Sara", "mark": 92}]</pre>
plain lines — you split them yourself	rows and columns, with a header line	nested structure — lists and dicts

Figure 3.1 The same three records stored as plain text, as CSV and as JSON

Format	Good for	Weakness
.txt	Logs, notes, anything line-based and simple	No structure — you must split and convert every line yourself
.csv	Tables: rows and columns, one record per line	Flat only; everything read back is a string
.json	Nested data — lists inside dictionaries inside lists	Larger files; a single stray character makes the whole file unreadable

Looking ahead: in Lab 08 you will load CSV files into Pandas with a single line. This lab is about what that line is doing underneath, which is what you will need the moment a file is malformed.

4. Opening a File: `open()`, `with`, and the Modes

`open()` takes a filename and a **mode**, and returns a file object. The file object must be closed afterwards, or your writes may never reach the disk. Doing that by hand is easy to get wrong, because an error in the middle of the work skips the `close()` altogether.

```
f = open('marks.txt', 'r')    # the fragile way
data = f.read()
f.close()                     # skipped entirely if read() raises
```

The `with` statement solves this. It closes the file the moment the block ends, whether the block finished normally or raised an exception, and it is the only form you should use in this course.

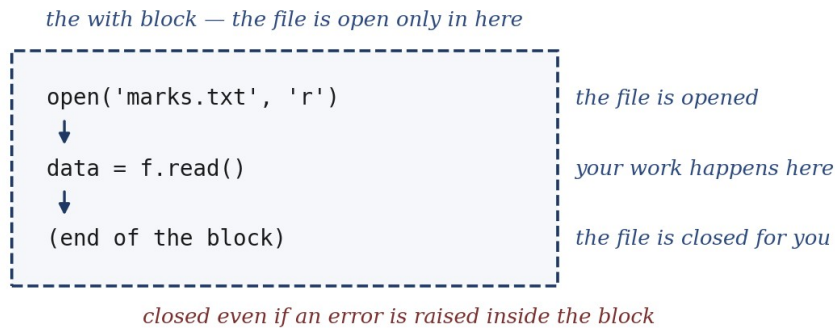


Figure 4.1 A file object opened by with is closed automatically at the end of the block

```

with open('marks.txt', 'r') as f:
    data = f.read()

print(f.closed)          # True - it was closed on the way out

```

4.1 The Modes

Mode	Meaning	If the file exists	If it does not
'r'	Read (the default)	Opens it	FileNotFoundError
'w'	Write	Erases it completely	Creates it
'a'	Append	Adds at the end	Creates it
'x'	Create only	FileExistsError	Creates it
'r+'	Read and write	Opens it, keeps the content	FileNotFoundError

The most expensive mistake in this lab: opening a file with 'w' when you meant 'a' erases it without asking and without any way back. When you intend to add to a file, check the mode twice.

A mode may also carry b for **binary** — 'rb', 'wb' — which is how images and other non-text files are read. Text files also take an encoding; use encoding='utf-8' whenever the data may contain non-English characters.

```

with open('urdu.txt', 'w', encoding='utf-8') as f:
    f.write("سلام\n")

```

5. Reading Text Files

For the examples below, assume a file marks.txt has already been created with three lines.

```

with open('marks.txt', 'w') as f:
    f.write("Ali 78\nSara 92\nBilal 65\n")

```

5.1 Three Ways to Read

```

# 1. the whole file as one string
with open('marks.txt') as f:
    content = f.read()
print(repr(content))

# 2. a list of lines - each one still carries its newline
with open('marks.txt') as f:

```

```

    lines = f.readlines()
    print(lines)

# 3. one line at a time - the memory-friendly way
with open('marks.txt') as f:
    for line in f:
        print(repr(line))

```

Output

```

'Ali 78\nSara 92\nBilal 65\n'
['Ali 78\n', 'Sara 92\n', 'Bilal 65\n']
'Ali 78\n'
'Sara 92\n'
'Bilal 65\n'

```

Method	Returns	Use it when
<code>f.read()</code>	The whole file as one string	The file is small and you need it all at once
<code>f.readline()</code>	The next single line	You are stepping through deliberately
<code>f.readlines()</code>	A list of lines	You want the line count, or random access
<code>for line in f:</code>	Each line in turn	Always, for large files — nothing extra is held in memory

5.2 Cleaning What You Read

Every line read from a text file ends with `\n`, and every value read from a text file is a **string**. Stripping and converting is not optional — it is the normal first step.

```

total = 0
count = 0

with open('marks.txt') as f:
    for line in f:
        line = line.strip()          # remove the newline and any spaces
        if not line:                # skip blank lines
            continue
        name, mark = line.split()    # split on whitespace
        total += int(mark)           # convert the text into a number
        count += 1
        print(name, "scored", mark)

print("average:", round(total / count, 2))

```

Output

```

Ali scored 78
Sara scored 92
Bilal scored 65
average: 78.33

```

A silent bug: `int("78\n")` happens to work, but `int("78 ")` with a trailing space does too, while `"78" == "78\n"` is `False`. Always `strip()` before you compare or convert.

6. Writing Text Files

`write()` takes a string and adds exactly that string — it does **not** add a newline for you, and it will not accept a number.

```
marks = {'Ali': 78, 'Sara': 92, 'Bilal': 65}

with open('report.txt', 'w') as f:          # 'w' - starts from empty
    f.write("MARKS REPORT\n")
    f.write("=" * 20 + "\n")
    for name, mark in marks.items():
        f.write(name + ", " + str(mark) + "\n")    # str() is required

with open('report.txt', 'a') as f:          # 'a' - adds to the end
    f.write("end of report\n")

with open('report.txt') as f:
    print(f.read())
```

Output

```
MARKS REPORT
=====
Ali,78
Sara,92
Bilal,65
end of report
```

`writelines()` writes a list of strings, again without adding newlines, so the newlines must already be in the strings. An f-string is usually the tidiest way to build each line.

```
rows = [f"{name},{mark}\n" for name, mark in marks.items()]

with open('report2.txt', 'w') as f:
    f.writelines(rows)

print(open('report2.txt').read())
```

Output

```
Ali,78
Sara,92
Bilal,65
```

7. Exception Handling

An **exception** is Python's way of reporting that something went wrong. It stops the current work and travels up through the program looking for code prepared to deal with it; if nothing is, the program crashes and prints a traceback. File work raises exceptions constantly — a missing file, a locked file, a line that does not contain the number you expected — so the two topics belong in the same lab.

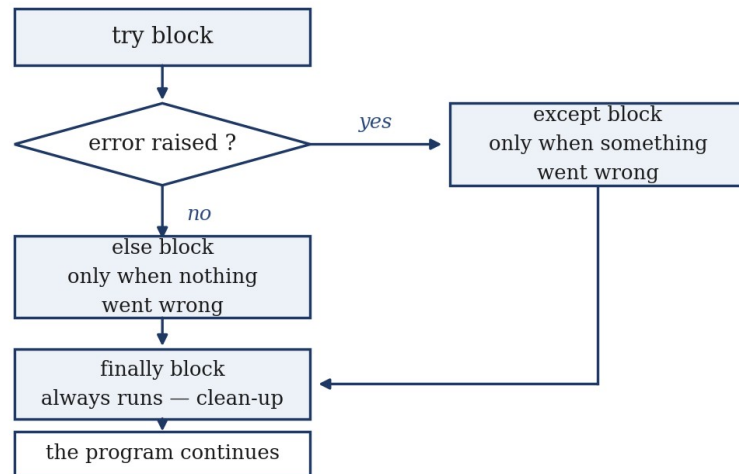


Figure 7.1 The path through try, except, else and finally

7.1 try and except

```

try:
    with open('missing.txt') as f:
        print(f.read())
except FileNotFoundError:
    print("that file does not exist - check the name and the folder")

print("the program carries on")

```

Output

```

that file does not exist - check the name and the folder
the program carries on

```

7.2 Catching the Right Exception

except: on its own catches **everything**, including your own typos and the `KeyboardInterrupt` you press to stop a runaway loop. Name the exception you expect. Several may be listed in one tuple, and several `except` blocks may follow one `try` — the first matching one runs.

```

def read_number(filename):
    try:
        with open(filename) as f:
            return int(f.read().strip())
    except FileNotFoundError:
        print("no such file:", filename)
    except ValueError:
        print("the file does not contain a whole number")
    except (PermissionError, IsADirectoryError) as err:
        print("could not open it:", err)
    return None

with open('good.txt', 'w') as f:
    f.write("42")
with open('bad.txt', 'w') as f:
    f.write("forty two")

print(read_number('good.txt'))
print(read_number('bad.txt'))
print(read_number('nowhere.txt'))

```

Output

```

42

```

```

the file does not contain a whole number
None
no such file: nowhere.txt
None

```

7.3 The Error Object, else and finally

`as err` gives you the exception object itself, which carries the message. `else` runs only when the `try` block finished without an exception — it keeps the protected section as small as possible. `finally` runs either way, and is for clean-up that must happen regardless.

```

def divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError as err:
        print("failed:", err)
        return None
    else:
        print("no error, the result is ready")
        return result
    finally:
        print("finally always runs")

print(divide(10, 4))
print(divide(10, 0))

```

Output

```

no error, the result is ready
finally always runs
2.5
failed: division by zero
finally always runs
None

```

Exception	Raised when
<code>FileNotFoundError</code>	Opening a file for reading that is not there
<code>PermissionError</code>	The operating system refuses access to the file
<code>ValueError</code>	<code>int('abc')</code> — the right type, an impossible value
<code>TypeError</code>	<code>'5' + 3</code> — the wrong type altogether
<code>KeyError</code> / <code>IndexError</code>	A missing dictionary key / a position past the end of a list
<code>ZeroDivisionError</code>	Division by zero
<code>json.JSONDecodeError</code>	The text being parsed is not valid JSON
<code>UnicodeDecodeError</code>	The file's encoding does not match the one you assumed

7.4 Raising Your Own

`raise` reports a problem your own code has detected — you used it in Lab 05 to reject a negative balance. A class derived from `Exception` gives that problem a name of its own, so callers can catch exactly it.

```

class EmptyFileError(Exception):
    """Raised when a data file contains no usable rows."""

    def load_marks(filename):

```

```

with open(filename) as f:
    rows = [line.strip() for line in f if line.strip()]
if not rows:
    raise EmptyFileError(filename + " contains no rows")
return rows

open('empty.txt', 'w').close()

try:
    load_marks('empty.txt')
except EmptyFileError as err:
    print("EmptyFileError:", err)

```

Output

```
EmptyFileError: empty.txt contains no rows
```

Ask forgiveness, not permission: checking if `os.path.exists(f)` before opening does not make the open safe — the file can vanish between the check and the open, and the check says nothing about permissions. Wrapping the open in `try / except` is both shorter and more reliable.

8. Where Your Files Live: Paths

A bare filename such as `'marks.txt'` is a **relative** path: it is looked for in the folder the program is running from, which for a notebook is the folder the notebook sits in. Getting this wrong is the single most common cause of `FileNotFoundError`.

```

import os

print(os.getcwd())           # where 'marks.txt' would be looked for
print(os.path.exists('marks.txt'))
print(os.path.abspath('marks.txt'))  # the full path it resolves to
print(os.listdir('.')[5])      # what is actually in this folder

```

Windows paths contain backslashes, which are also Python's escape character, so `'C:\\data\\new.txt'` must be written as a raw string `r'C:\data\new.txt'` or with forward slashes. Better still, build the path with `os.path.join()`, which inserts the separator the current operating system uses.

```

import os

path = os.path.join('data', 'reports', 'marks.txt')
print(path)  # data/reports/marks.txt on Linux

os.makedirs('data/reports', exist_ok=True)  # create the folders if needed

with open(path, 'w') as f:
    f.write("Ali 78\n")

print(os.path.exists(path), os.path.getsize(path), "bytes")
print(os.path.splitext(path))

```

Output

```

data/reports/marks.txt
True 7 bytes
('data/reports/marks', '.txt')

```

9. CSV Files

A CSV file is a table stored as text: one record per line, fields separated by commas, and usually a header line naming the columns. You could split each line yourself, and the old way of doing it is worth seeing once — but a value containing a comma, such as "Khan, Ali", breaks that approach immediately. The `csv` module handles the quoting rules for you.

9.1 Writing and Reading Rows

```
import csv

rows = [['name', 'subject', 'mark'],
        ['Ali', 'Physics', 78],
        ['Sara', 'Maths', 92],
        ['Khan, Ali', 'Chemistry', 65]]    # note the comma inside a value

with open('marks.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(rows)                # writerow() for a single row

with open('marks.csv', newline='') as f:
    reader = csv.reader(f)
    header = next(reader)                 # pull the header off first
    print("columns:", header)
    for row in reader:
        print(row, "-> mark as a number:", int(row[2]))
```

Output

```
columns: ['name', 'subject', 'mark']
['Ali', 'Physics', '78'] -> mark as a number: 78
['Sara', 'Maths', '92'] -> mark as a number: 92
['Khan, Ali', 'Chemistry', '65'] -> mark as a number: 65
```

Two details that matter: `newline=''` in the `open()` call stops blank lines appearing between rows on Windows, and every field read back is a **string** — convert with `int()` or `float()` before doing arithmetic.

9.2 DictReader and DictWriter

Reading rows as dictionaries keyed by the header is almost always clearer: `row['mark']` says what it means, while `row[2]` has to be counted out by the reader.

```
import csv

with open('marks.csv', newline='') as f:
    reader = csv.DictReader(f)            # the header becomes the keys
    total = 0
    for row in reader:
        print(row['name'], "->", row['mark'])
        total += int(row['mark'])
    print("total:", total)

records = [{'name': 'Hina', 'subject': 'Maths', 'mark': 88},
           {'name': 'Usman', 'subject': 'Physics', 'mark': 54}]

with open('marks2.csv', 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=['name', 'subject', 'mark'])
    writer.writeheader()                  # writes the header line for you
    writer.writerows(records)

print(open('marks2.csv').read())
```

Output

```

Ali -> 78
Sara -> 92
Khan, Ali -> 65
total: 235
name,subject,mark
Hina,Maths,88
Usman,Physics,54

```

10. JSON Files

JSON — JavaScript Object Notation — is text that represents nested data. It is what almost every web API speaks, which is why this section comes immediately before the API section. Python's `json` module converts in both directions.

Python	JSON	Python again after a round trip
dict	object { }	dict
list, tuple	array []	list — a tuple comes back as a list
str	string	str
int, float	number	int, float
True / False / None	true / false / null	True / False / None
a set, a Student object	— not supported —	TypeError when dumping

10.1 Four Functions

The pair with `s` works on **strings**; the pair without works on **files**. That is the whole distinction, and it is the thing most often got wrong.

Function	Direction	Works on
<code>json.dump(obj, f)</code>	Python → JSON	An open file
<code>json.load(f)</code>	JSON → Python	An open file
<code>json.dumps(obj)</code>	Python → JSON	Returns a string
<code>json.loads(text)</code>	JSON → Python	Reads from a string

```

import json

students = [
    {'name': 'Ali', 'marks': [78, 45, 90], 'active': True},
    {'name': 'Sara', 'marks': [88, 92, 95], 'active': False},
]

with open('students.json', 'w') as f:
    json.dump(students, f, indent=2)    # indent makes it readable

with open('students.json') as f:
    loaded = json.load(f)

print(type(loaded), len(loaded))
print(loaded[0]['name'], loaded[0]['marks'])

```

```
print(loaded[1]['active'], type(loaded[1]['active']))
```

Output

```
<class 'list'> 2
Ali [78, 45, 90]
False <class 'bool'>
```

```
import json

text = json.dumps({'name': 'Ali', 'cgpa': 3.4})    # to a string
print(text, type(text))

back = json.loads(text)                          # and from a string
print(back['cgpa'] + 0.1)
```

Output

```
{"name": "Ali", "cgpa": 3.4} <class 'str'>
3.5
```

10.2 When JSON Goes Wrong

A JSON file is all or nothing: one missing bracket and the whole file fails to parse. Two failures are worth meeting deliberately.

```
import json

with open('broken.json', 'w') as f:
    f.write('{"name": "Ali", }')    # a trailing comma - not valid JSON

try:
    with open('broken.json') as f:
        json.load(f)
except json.JSONDecodeError as err:
    print("JSONDecodeError:", err)

try:
    json.dumps({'subjects': {'Physics', 'Maths'}})    # a set cannot be stored
except TypeError as err:
    print("TypeError:", err)
```

Output

```
JSONDecodeError: Expecting property name enclosed in double quotes: line 1 column 17
(char 16)
TypeError: Object of type set is not JSON serializable
```

Keys become strings: `json.dump({1: 'a'}, f)` stores the key as "1", and it comes back as the string '1', not the number 1. If your dictionary is keyed by numbers, convert them back after loading.

11. Web APIs and the requests Library

An **API** — application programming interface — is a service that another program can call. A **web API** is one you reach over HTTP: your program sends a request to a URL, and the server sends back a response whose body is usually JSON. From Python's point of view it is the same work you have just done with files, except that the data arrives from another machine and any step of it may fail.

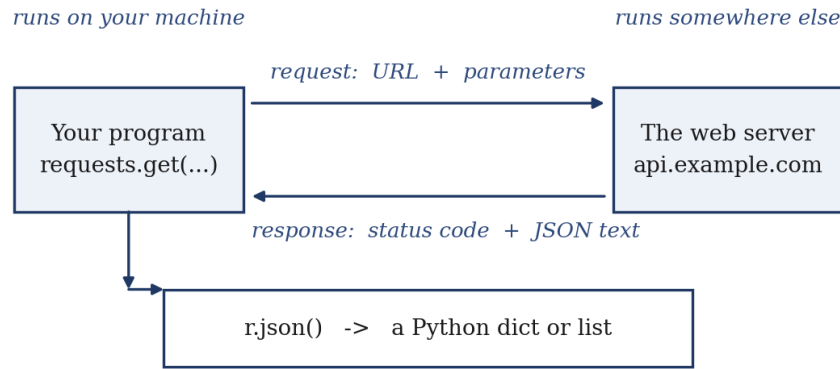


Figure 11.1 A request goes out, a response comes back, and `.json()` turns it into Python objects

11.1 A First Request

The `requests` library is not part of the standard library; install it once with `pip install requests`.

```
import requests

url = "https://api.github.com/repos/python/cpython"
response = requests.get(url, timeout=10)

print(response.status_code)          # 200 means success
print(response.headers['Content-Type'])

data = response.json()                # JSON text -> a Python dictionary
print(type(data))
print(data['name'], "-", data['language'])
```

Output

```
200
application/json; charset=utf-8
<class 'dict'>
cpython - Python
```

Always set a timeout: without `timeout=`, a request to an unresponsive server can hang your program indefinitely. Ten seconds is a reasonable default in this lab.

11.2 The Status Code

The server answers every request with a three-digit code. Checking it before touching the body is not optional: an error page is not JSON, and calling `.json()` on it raises.

Code	Family	Meaning
200	2xx — success	The request worked; the body holds your data
301 302	3xx — redirect	The resource lives elsewhere; <code>requests</code> follows these for you
400	4xx — your mistake	The request was malformed
401 403	4xx — your mistake	Authentication missing or refused — many APIs need a key
404	4xx — your mistake	No such resource — usually a typo in the URL
429	4xx — your mistake	Too many requests; you are being rate limited

Code	Family	Meaning
500 503	5xx — their mistake	The server failed or is unavailable; retrying later may work

```
import requests

response = requests.get("https://api.github.com/repos/python/does-not-exist",
                        timeout=10)
print(response.status_code)
print(response.ok)                # True only for 2xx

if response.status_code == 200:
    print(response.json()['name'])
else:
    print("the request failed, so the body was not parsed")
```

Output

```
404
False
the request failed, so the body was not parsed
```

11.3 Query Parameters

Most APIs are configured through parameters added to the URL after a ?. Build them as a dictionary and let requests assemble and escape the URL — hand-built URLs break as soon as a value contains a space or an ampersand.

```
import requests

params = {'q': 'language:python stars:>50000', 'sort': 'stars', 'per_page': 3}
response = requests.get("https://api.github.com/search/repositories",
                        params=params, timeout=10)

print(response.url)
data = response.json()
for repo in data['items']:
    print(repo['full_name'], "-", repo['stargazers_count'], "stars")
```

11.4 Handling Failure Properly

A network call can fail in ways a file never does: no internet, a refused connection, a server that never answers, a body that is not JSON. A request you intend to rely on needs all of these covered.

```
import requests, json

def fetch_repo(full_name, timeout=10):
    """Return the repository dictionary, or None if anything goes wrong."""
    url = "https://api.github.com/repos/" + full_name
    try:
        response = requests.get(url, timeout=timeout)
        response.raise_for_status()    # turns 4xx and 5xx into an exception
        return response.json()
    except requests.exceptions.Timeout:
        print("the server took too long to answer")
    except requests.exceptions.ConnectionError:
        print("no network connection")
    except requests.exceptions.HTTPError as err:
        print("the server refused the request:", err.response.status_code)
    except json.JSONDecodeError:
```

```

        print("the reply was not JSON")
        return None

repo = fetch_repo("python/cpython")
print(repo['stargazers_count'] if repo else "nothing to show")

```

Be a good citizen: a public API is someone else's machine. Never put a bare `requests.get()` inside a fast loop — add a short `time.sleep()` between calls, request only what you need, and cache the reply to a file while you are developing so that a hundred test runs cost one request.

12. A Complete Pipeline: Fetch, Parse, Store

The three halves of this lab fit together into the shape you will use for the rest of the course: fetch the data, handle whatever goes wrong, keep the raw reply as JSON, and write the tidy summary as CSV. Notice that the functions do not print — only `main()` does, exactly as in Lab 04.

```

import csv, json, requests

def fetch(names):
    """Return a list of repository dictionaries, skipping any that fail."""
    repos = []
    for name in names:
        try:
            r = requests.get("https://api.github.com/repos/" + name, timeout=10)
            r.raise_for_status()
            repos.append(r.json())
        except requests.exceptions.RequestException as err:
            print("skipped", name, "-", err)
    return repos

def save_raw(repos, filename='raw.json'):
    """Keep the untouched reply so the API need not be called again."""
    with open(filename, 'w') as f:
        json.dump(repos, f, indent=2)

def summarise(repos):
    """Pull out only the fields we care about."""
    return [{'name': r['full_name'],
              'stars': r['stargazers_count'],
              'language': r['language'] or 'unknown'} for r in repos]

def save_csv(rows, filename='summary.csv'):
    with open(filename, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=['name', 'stars', 'language'])
        writer.writeheader()
        writer.writerows(rows)

def main():
    repos = fetch(["python/cpython", "pallets/flask", "no/such-repo"])
    if not repos:
        print("nothing was fetched")
        return
    save_raw(repos)
    rows = summarise(repos)

```

```

save_csv(rows)
for row in sorted(rows, key=lambda r: r['stars'], reverse=True):
    print(row['name'], row['stars'])

main()

```

13. Common Errors

Message or symptom	Usual cause
FileNotFoundError: [Errno 2] ...	The path is relative to the wrong folder — print <code>os.getcwd()</code> and check.
The file is empty after writing	The program ended without closing it; use <code>with</code> .
The file's old contents vanished	Mode <code>'w'</code> was used where <code>'a'</code> was meant.
TypeError: write() argument must be str	A number was passed to <code>write()</code> ; wrap it in <code>str()</code> .
Every value read back is text	That is how text files work — convert with <code>int()</code> or <code>float()</code> .
A blank line between every CSV row	<code>newline=''</code> was left out of the <code>open()</code> call.
json.JSONDecodeError: Expecting value	The file is empty, is not JSON, or is a <code>.txt</code> written with <code>str(list)</code> .
TypeError: Object of type set is not JSON serializable	JSON has no set; convert it to a list first.
AttributeError: 'str' object has no attribute 'get'	<code>json.load</code> was given a string — use <code>json.loads</code> for that.
ModuleNotFoundError: No module named 'requests'	Run <code>pip install requests</code> .
<code>.json()</code> raises on an error page	The status code was not checked before parsing the body.

14. Quick Reference

Form	Meaning	Example
<code>with open(p) as f:</code>	Open and close safely	Mode <code>'r'</code> by default
<code>'r' 'w' 'a' 'x'</code>	Read / overwrite / append / create	<code>open(p, 'a')</code>
<code>f.read()</code> <code>f.readlines()</code>	Whole file / list of lines	<code>for line in f:</code> for large files
<code>f.write(s)</code> <code>f.writelines(L)</code>	Write text — newlines are yours to add	<code>f.write(str(n) + '\n')</code>
<code>line.strip().split(',')</code>	Clean and split a text line	The standard first step
<code>os.path.join(a, b)</code>	Build a portable path	<code>os.path.exists(p)</code>
<code>csv.reader(f)</code> <code>csv.writer(f)</code>	Rows as lists	<code>newline=''</code> in <code>open()</code>

Form	Meaning	Example
<code>csv.DictReader(f)</code>	Rows as dictionaries keyed by the header	<code>row['mark']</code>
<code>csv.DictWriter(f, fieldnames=...)</code>	Write dictionaries	<code>writer.writeheader()</code>
<code>json.dump(obj, f)</code> <code>json.load(f)</code>	Python ↔ JSON, through a file	<code>indent=2</code> to format
<code>json.dumps(obj)</code> <code>json.loads(s)</code>	Python ↔ JSON, through a string	The <code>s</code> is for string
<code>try / except / else / finally</code>	Handle, continue, clean up	<code>except ValueError as err:</code>
<code>raise ValueError(msg)</code>	Report a problem yourself	<code>class MyError(Exception):</code>
<code>requests.get(url, params=..., timeout=...)</code>	Send a GET request	<code>pip install requests</code>
<code>r.status_code</code> <code>r.ok</code> <code>r.json()</code>	Check, then parse	<code>r.raise_for_status()</code>

15. Class Tasks

These five tasks are to be completed **during the lab session** and demonstrated to the instructor before you leave. Work in the accompanying notebook `Lab06_Files_Exceptions_and_APIs.ipynb`, in the section titled **Class Tasks**. Every task must survive being run twice in a row, and every file operation must be wrapped in `with` and protected by a specific `except`.

C1. A Text-File Report — reading, writing and cleaning. Create a file `students.txt` from your own code, containing at least six lines of the form `name,subject,mark`. Then write a program that reads it back line by line and prints: the number of lines, the number of **valid** records, the overall average mark, the highest and lowest scorer, and a list of every line it had to reject. Your reader must survive a file that contains a blank line, a line with a missing field, and a line whose mark is not a number — add all three to the file deliberately and catch each failure with the right exception rather than with a bare `except`.

Finish by writing the summary to `report.txt` in append mode, so that running the program a second time adds a second report instead of erasing the first.

C2. A Robust Reader — exception handling. Write a function `load_marks(filename)` that returns a list of `(name, mark)` tuples and handles every failure separately: `FileNotFoundError` (report it and return an empty list), `ValueError` (skip that single line and keep going, counting how many were skipped), and `PermissionError`. Define your own exception `EmptyFileError(Exception)` and raise it when the file parses correctly but contains no usable rows. Use `finally` to print how many lines were examined, whatever happened.

Call the function four times to prove every path works: on a good file, on a file containing bad lines, on an empty file, and on a filename that does not exist. All four calls must run one after another without the notebook stopping.

C3. Marks to CSV and Back — the `csv` module. Build a list of at least six student dictionaries with the keys `roll`, `name`, `subject` and `mark`, and write them to `marks.csv` with `DictWriter`, header included. Then, reading the file back with `DictReader` — and never with `split(',')` — print the class average, the number of students per subject using a dictionary, and the top scorer in each subject. Finally write a second file `toppers.csv` containing only the top scorer of each subject.

Include one student whose name contains a comma, such as `"Khan, Ali"`, and show that it survives the round trip intact. Explain in a comment why splitting on commas by hand would have broken.

C4. A JSON Address Book — nested data. Build a dictionary of at least four people, each mapping a name to a dictionary holding age, a list of subjects and a nested contact dictionary with email and city. Save it to `people.json` with `indent=2`, load it back, and print: everyone's name and city, the person with the highest age, and anyone who shares that age. Then add a new person, save the file again, and reload it to prove the addition persisted.

Also demonstrate two failures inside `try / except`: loading a deliberately corrupted JSON file, and attempting to `json.dumps` a dictionary containing a set. Print the exception message in both cases rather than letting the notebook stop.

C5. A Live API Client — requests end to end. Using the public API `https://api.github.com/users/<username>`, write a function `fetch_user(username)` that returns the user dictionary or `None`, checking the status code and catching `Timeout`, `ConnectionError` and `HTTPError` separately. Call it for three usernames, one of which must not exist. For every user that was found, print the login name, the number of public repositories and the account creation date; save the full replies to `users.json`; and write a summary table of the found users to `users.csv`.

If the lab machines have no internet access, the notebook provides a small offline sample reply — use it so that your parsing, saving and error handling can still be demonstrated in full. Your code must not change between the online and the offline case apart from where the data comes from.

Assessment of class tasks: you must be able to explain any line of your code — in particular which exception each `except` is guarding against, why a given file mode was chosen, and what would happen if the program were interrupted halfway through a write. A working program that you cannot explain earns no marks.

16. Home Tasks

Attempt all the questions below. Write each program in a **separate .py file**, handle every exception that can reasonably occur, and submit according to the naming convention in Section 17. Where a question says **make the logic yourself**, you may use only the standard file, csv and json operations — no other library.

- Q1.** Write a program that reads a text file and prints the number of characters, the number of words and the number of lines in it. Handle every possible exception.
- Q2.** Write a program that reads a text file, searches for a word or phrase given by the user, replaces every occurrence with another word or phrase, and writes the modified content back to the same file. Report how many replacements were made.
- Q3.** Take two lists of equal length from the user. Build a dictionary in which the first element of the first list is a key and the first element of the second list is its value, and so on, then store that dictionary in a text file. Make the logic yourself — do not use `zip()` or any library. Handle every possible exception.
- Q4.** Take an employee's biodata from the user — name, CNIC number, age and salary — and save it to a text file. Then append a contact number to the same file, and finally read the whole file back and print it. Explain in a comment what would have happened had you used mode `'w'` for the second step.
- Q5.** Ask the user for name-and-age pairs and store them in a dictionary such as `{ 'Ali': 23, 'Saad': 24, 'Salman': 15, 'Shams': 25, 'Sadiq': 46, 'Hammad': 23 }`. Save it as a JSON file, load it back, and print the name of the oldest person, and also print anyone who shares an age with somebody else. Work out the maximum and the duplicates with your own loops, not with `max()` or `Counter`.
- Q6.** Write a function that asks the user for a sentence and appends it to a file named `questions.txt` only if it is a question. Call the function in a loop until the user types `quit`, then print the whole file. Handle every possible exception.
- Q7.** A file `replacement_needed.txt` contains one sentence with a single mistake: one letter must be replaced by another for the sentence to make sense. Read the file, work out and perform the replacement with your own logic, and write the corrected sentence to a new file. Put your code in a function and call it.

- Q8.** Take two numbers from the user and divide them. Handle `ZeroDivisionError` when the second number is zero, and `ValueError` when the input is not a number. Keep asking until both inputs are valid, then report the result.
- Q9.** Write a program that reads a CSV file of students — `roll, name, subject, mark` — with `DictReader`, and writes two new CSV files: one containing only the students who passed (50 or more) and one containing only those who failed. Print how many rows went into each file, and handle a missing file and a malformed row.
- Q10.** Write a program that converts between formats: read `students.csv` and write the same data as `students.json` (a list of dictionaries), then read that JSON back and write it out as a plain text report with one line per student. Verify that the number of records is the same at every stage.
- Q11.** Write a simple log analyser. Create a text file in which each line is `date time level message`, with levels such as `INFO`, `WARNING` and `ERROR`. Read it and report: how many lines carried each level, the first and last `ERROR` message, and every line belonging to a date entered by the user. Handle lines that do not match the expected shape.
- Q12.** Using `requests`, fetch a public API of your choice that needs no key — for example `https://api.github.com/users/<username>` or `https://api.coindesk.com/v1/bpi/currentprice.json` — and write a program that checks the status code, parses the JSON, prints three interesting fields, and saves the full reply to a file. Catch `Timeout`, `ConnectionError` and `HTTPError` separately, and make the program print a clear message (rather than crashing) when the machine is offline.

17. Submission and Assessment

Follow the naming convention below so that your work can be identified and graded correctly:

- **Class tasks:** completed in the notebook `RollNo_Lab06_ClassTasks.ipynb` and demonstrated before the end of the session. Run **Kernel ► Restart & Run All** before you call the instructor.
- **Home tasks:** one script per question, named `RollNo_Lab06_Qn.py` — for example `22K-1234_Lab06_Q7.py`.
- Submit the **data files** your programs create or read alongside the code, in the same folder.
- Place everything for this lab in a single folder and compress it before uploading.
- Every file must begin with a comment giving your name, roll number and the question number.
- Code that does not run will not be marked. Test your program before calling the instructor.

Assessment Criterion	Weight
Correctness of output, including on malformed and missing files	30%
Class tasks completed and demonstrated in the session	20%
Correct file handling: <code>with</code> , the right mode, clean parsing	20%
Exception handling: specific exceptions, no bare <code>except</code>	20%
Readability: functions, naming, comments	10%

End of Lab 06